When Agents Wait: Memory Bottlenecks of Agent Serving on an Edge LLM Server

Kai-Cheng Liu
University of California San Diego
CSE 291A, Fall 2026 — Phase 1 report

***Abstract*—LLM agents call the model repeatedly, pause for tool calls, and return with a longer conversation each time. We study how one edge LLM server should admit such agents under a fixed memory budget. On llama.cpp with a 3B model, we build a seeded multi-step agent workload and measure a FIFO scheduler with a fixed concurrency limit K on an Apple M2 Mac, replicated on an AWS L4 GPU slice. The best K equals the server's slot count on both machines. Admitting more agents than slots evicts their cached context between steps: prompt-cache reuse falls from 65% to 17–19%, and p95 latency rises 2.0–2.4× on the Mac but only 1.4× on the GPU, which recomputes context 4× faster. Retaining context recovers most of the loss, motivating Phase 2: memory-bounded retention of agent context across tool calls.**

I. Introduction

LLM agents run a loop [1]: call the model, wait for a tool such as a search or a code run, then call the model again with the whole conversation so far. Prompts grow at every step, and between steps the agent leaves the model idle. A server keeps each active conversation in its key–value (KV) cache, whose size is fixed by memory. Admitting few agents leaves the hardware idle during tool waits; admitting many overflows that memory, so returning agents find their context evicted and must recompute it. The dilemma is sharpest on edge servers, where one machine with a few gigabytes serves several agents.

Phase 1 asks: *under a fixed memory budget, how do concurrency, context growth and tool waits affect completed agent work, and what is the dominant bottleneck of a FIFO baseline?* We contribute (1) a seeded agent workload and an instrumented harness for llama.cpp, (2) a tuned FIFO baseline on two hardware platforms, (3) evidence, from mechanism and a causal test, that the bottleneck is context eviction once K exceeds the server's slots, and (4) a Phase 2 plan (timeline in the appendix).

II. Background and Related Work

A transformer server stores the keys and values of processed tokens; for Llama-3.2-3B this costs 112 KiB per token in f16. llama.cpp [2] splits a fixed KV allocation (`-c` tokens) evenly among `-np` *slots*, each serving one request. A slot keeps its last prompt, so a returning conversation whose prefix is still there reuses it (*prompt caching*); otherwise its whole history is processed again. vLLM [3] instead pages one shared pool, and SGLang [4] shares prefixes through a radix tree. For agents, InferCept [5] chooses per request whether to preserve, discard or swap context during a tool call, and Continuum [6] pins it for a time-to-live. Both target large GPU servers; we study a single edge machine with a hard memory budget.

III. Experimental Setup

Platforms. The primary platform is an Apple M2 Mac mini (24 GB unified memory, Metal); the replication runs on an AWS g6f.2xlarge with an NVIDIA L4-6Q (a quarter of an L4, 6 GiB, CUDA). Both run llama.cpp b11803 with Llama-3.2-3B-Instruct Q4_K_M (SHA-256 checked) and 4 slots of 4,096 tokens (`-c 16384 -np 4`), allocating 4.1 GB (weights 2.2, KV 1.8, compute 0.2) of a 6 GB budget. Micro-benchmarks show different balances: prompt processing runs at 365–427 vs. 1,463–1,651 tokens/s (3.4–4.2× faster on AWS), single-request generation at 37.5 vs. 23.3 tokens/s.

Workload. A seeded generator produces four task classes sharing a 200-token system prefix: *chat* (30%, 1 step, 150–400-token context), *RAG* (20%, 1 step, 2,000–3,000 tokens), *short agents* (30%, 3 steps, 0.5–2 s tool calls returning 100–300 tokens) and *long agents* (20%, 6 steps, 1–4 s tool calls returning 200–400 tokens). Agent steps resend the full history, so contexts grow 200–600 tokens per step. Tasks arrive as a Poisson process at 0.05, 0.1 or 0.2 tasks/s (light load to about 3× overload); each run has 30 tasks, each configuration three seeds.

Baseline and controls. FIFO admits tasks up to a limit K; a task keeps its admission through tool calls. Every step generates a fixed number of tokens, each run starts a fresh server after two warm-up requests, and llama.cpp's host-RAM prompt cache is off so memory equals the allocation. We report p95 task latency, throughput, prompt-cache reuse, failures by cause and memory. Failed tasks are kept; runs disturbed by a leftover server were excluded and rerun.

IV. Baseline: FIFO with Fixed K

Fig. 1 summarizes 90 runs. On both machines latency is lowest when K equals the slot count. Under overload, $K = 4$ cuts p95 latency 2.0–2.4× relative to $K = 1$ on the Mac and 3.4–4.0× on AWS, where single-request generation is slow (6 of 90 AWS tasks at $K = 1$, 0.2 tasks/s,

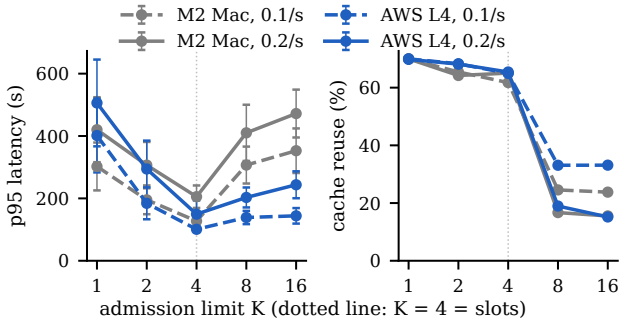


Fig. 1. FIFO baseline over K on both machines (mean of 3 seeds; bars: ± 1 std). Latency is lowest at $K = 4$, the slot count; above it, cached context is evicted and reuse collapses on both machines.

TABLE I
GOING FROM $K = 4$ TO $K = 8$ AT 0.2 TASKS/S (MEAN OF 3 SEEDS)

	M2 Mac	AWS L4-6Q
Prompt-cache reuse	65% → 17%	65% → 19%
Median prompt time / request	1.17 → 3.55 s	0.37 → 0.96 s
Generation / request	12.3 → 8.9 tok/s	12.4 → 10.8 tok/s
p95 time to first token	9 → 39 s	2 → 19 s
p95 task latency	205 → 410 s	149 → 203 s

exceeded the 600 s limit). Raising K beyond the slots makes every metric worse: from $K = 4$ to $K = 8$ at 0.2 tasks/s, p95 rises from 205 to 410 s on the Mac and from 149 to 203 s on AWS. Throughput falls 30% on the Mac (0.075 to 0.053 tasks/s) and 6% on AWS (0.090 to 0.084). In almost every configuration 1.7 of 30 tasks fail because a long agent outgrows its slot.

V. THE BOTTLENECK: CONTEXT EVICTION

Mechanism. With $K > 4$, more conversations are active than slots. A task returning from a tool call usually finds its slot reassigned and its context overwritten, so it reprocesses its whole history. Table I shows reuse collapsing equally on both machines while the rebuild cost differs: each request spends 2.4 s more on prompt processing on the Mac but 0.6 s on AWS, and the recomputations slow co-batched generation by 28% vs. 12%. The penalty is roughly the reuse lost (set by the scheduler) times the recomputation cost (set by the hardware).

Causal test. If eviction is the cause, keeping evicted context should remove the penalty. Fig. 2 shows $K = 8$ on the Mac with llama.cpp's host-RAM prompt cache (`-cache-ram`): a 1 GB host cache raises reuse from 24% to 53% and cuts p95 from 314 to 199 s within a 6 GB budget; an 8 GB cache restores 70% reuse and matches the best fixed K (128 vs. 127 s) but needs 11.8 GB. Disabling prompt caching is worse still (375 s).

Memory split. At a fixed KV budget, smaller slots fail more tasks (14 of 30 at 2,048 tokens, 9.3 at 3,072, 1.7 at 4,096). An 8-bit KV cache halves KV memory (952 vs. 1,792 MiB) at unchanged speed, so 8 slots of 4,096 tokens fit in the memory of 4; at $K = 8$ reuse returns to 63%, throughput rises 42% and p95 falls from 314 to 179 s. Its outputs matched f16 exactly on 30 long-context retrieval and 20 short-answer prompts; 6 of 20 long free-form answers diverged after a median of 56 tokens into equally fluent text (a repeated f16 run was identical).

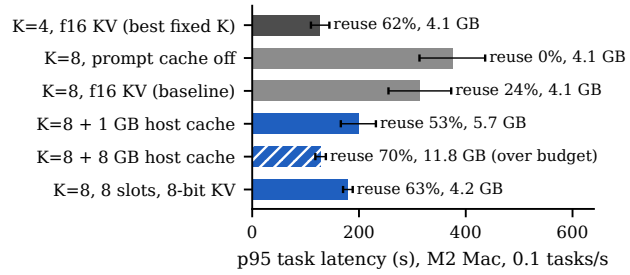


Fig. 2. $K = 8$ on the M2 Mac at 0.1 tasks/s under different memory configurations (mean ± 1 std of 3 seeds). Memory is weights plus measured footprint. Retaining context recovers latency; the hatched configuration exceeds the budget.

Limitations. The workload is synthetic (filler text; tool delays and step counts chosen, not measured). Slot eviction is specific to fixed-slot servers such as llama.cpp; a paged pool [3] degrades differently. p95 over 30 tasks is noisy, so we claim only differences above one standard deviation. The Mac's budget is enforced by configuration.

VI. PHASE 2 PLAN

Retention pays off, but a fixed host cache is either too small or exceeds the budget. Phase 2 will build *memory-bounded context retention*: admit more agents than slots and, when an agent enters a tool call, keep its context in the slot, park it on the edge machine's SSD, or drop it, weighing each platform's measured recomputation cost against the memory other agents need. Returning agents get free slots first. A probe shows parking is cheap: saving and restoring a 1,000–3,500-token slot through llama-server takes 0.2–0.5 s versus 2.6–9.7 s to recompute it on the Mac, with every token reused. Baselines are FIFO at $K = 4$, $K = 8$ with a 1 GB host cache, and 8 slots with 8-bit KV; a Continuum-style time-to-live is an ablation. Evaluation uses held-out seeds on both machines, plus longer tool calls, and reports p95 per task class, throughput, peak memory and save/restore overhead.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.
[2] ggml-org, "llama.cpp: LLM inference in C/C++," https://github.com/ggml-org/llama.cpp, build b11803 (commit a37cc461), 2026.

[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[4] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[5] R. Abhyankar, Z. He, V. Srivatsa, H. Zhang, and Y. Zhang, "InferCept: Efficient intercept support for augmented large language model inference," in *Proc. 41st International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 235, 2024, pp. 81–95.

[6] H. Li, R. He, Q. Mang, Q. Zhang, H. Mao, X. Chen, H. Zhou, H. Zhang, A. Cheung, J. Gonzalez, and I. Stoica, "Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live," arXiv:2511.02230, 2025.

APPENDIX
TIMELINE AND MILESTONES

Dates	Milestone
Oct 8–21	Topic and related work; harness, workload generator and mock server; llama.cpp and model pinned on the Mac ✓
Oct 22–24	Micro-benchmarks; pilot; K, retention and memory-split sweeps on the Mac (99 runs) ✓
Oct 25–27	AWS setup; K sweep and micro-benchmarks replicated (45 runs); 8-bit KV quality check; slot save/restore probe ✓
Nov 4	Phase 1 code, data and report submitted
Nov 5–14	Retention policy in the runner (keep / park on SSD / drop; returning-agent priority); time-to-live and always-park ablations; tests on dev seeds
Nov 15–21	Measure save/restore interference under load; tune the cost model per platform
Nov 22–30	Evaluation on held-out seeds (1000+), both machines, current mix and long-tool-call mix
Dec 1–10	Final report and code release